

Ejercicios Flash - Final EDA

Ruta de 1 hora: problemas con maxima transferencia + teoria para responder en papel

Tema	Problema	Por que entra
STATIC	Static RMQ	Sparse table: min/max/gcd idempotente, queries estaticas.
STRINGS	Suffix array + LCP	Base para buscar, contar, comparar y ordenar subcadenas.
STRINGS	Busqueda/Conteo de subcadenas	Patron P -> intervalo contiguo de sufijos con prefijo P.
STRINGS	Subcadenas distintas / LCS / ordenar subcadenas	Formulas con LCP y RMQ sobre LCP.
STATIC TREE	Imperial Roads	MST + max edge on path por LCA/HLD.
STATIC TREE	Fools and Roads	LCA + diferencia en arbol + postorder.
PRACTICA PROF	Subsecuencia en camino de arbol	HLD + posiciones por caracter para consumir patron.
PRACTICA PROF	Orden lexicografico de nodos de trie	DFS con hijos ordenados; prefix antes que extension.
PRACTICA PROF	Conteo de caminos con peso $\leq C$	Centroid decomposition + two pointers.
DYNAMIC	Offline dynamic connectivity	Segment tree sobre tiempo + DSU rollback.
DYNAMIC	Dynamic forest conceptual	Euler-tour tree para componente/subtree; Link-cut para path.
INTEGER	Predecessor	vEB/Y-fast/Fusion segun universo, espacio y modelo.
INTEGER	Integer sorting	Counting/radix/signature: romper barrera de comparaciones.

0. Selección de emergencia

Con 1 hora, no conviene intentar aprender todo linealmente. Hay que aprender patrones que se reciclan en preguntas teóricas. Si el examen trae 3 preguntas, una de strings confirmada, la prioridad real es: (1) suffix array/LCP y sus aplicaciones; (2) árboles estáticos con LCA/HLD; (3) dynamic connectivity offline o dynamic forest conceptual; (4) integer como respuesta teórica de modelos, predecessor y sorting.

La forma de responder en papel siempre debe ser: identificar objeto fijo/dinámico, escoger estructura, decir que guarda, transformar la consulta, probar el invariante y cerrar complejidad. Si solo escribes el nombre de la estructura, te faltan puntos.

1. Static RMQ - Sparse Table

Problema tipo: arreglo fijo A, muchas consultas $\min(l, r)$. Si no hay updates y la operación es idempotente como $\min/\max/\gcd$, sparse table es mejor que segment tree para teoría: preprocess $O(n \log n)$, query $O(1)$.

Definición: $st[k][i]$ guarda la respuesta del bloque $A[i..i+2^k-1]$. Transición: $st[k][i] = \min(st[k-1][i], st[k-1][i+2^{k-1}])$. Para query de longitud len , $k = \lfloor \log_2 len \rfloor$. Se combinan dos bloques de longitud 2^k : inicio y final. Para rango cerrado $[l, r]$, $len = r - l + 1$ y $ans = \min(st[k][l], st[k][r - 2^k + 1])$. Para rango medio abierto $[l, r]$, $len = r - l$ y $ans = \min(st[k][l], st[k][r - 2^k])$.

Correctitud: cada st es correcto por inducción sobre k porque el bloque de longitud 2^k se parte en dos mitades de 2^{k-1} . En una query, los dos bloques cubren todo el rango; si se solapan no importa porque $\min(x, x) = x$. Complejidad: $O(n \log n)$ memoria y preprocess, $O(1)$ query.

Pseudo: build logs; $st[0][i] = A[i]$; for k 1..LG: for i valid: $st[k][i] = \min(st[k-1][i], st[k-1][i+2^{k-1}])$; query(l, r): $len = r - l + 1$; $k = \lg len$; return $\min(st[k][l], st[k][r - 2^k + 1])$.

2. Suffix Array, Rank, LCP, Kasai

Sufijo: $S[i..n-1]$. Prefijo: $S[0..i]$ o prefijo de cualquier string. Suffix array SA ordena las posiciones i por el texto $S[i..]$. $rank[i]$ es el inverso: posición del sufijo i dentro de SA. $LCP[i]$ es el longest common prefix entre sufijos vecinos $SA[i-1]$ y $SA[i]$.

Manber-Myers: ordena por longitudes 1, 2, 4, 8, ... Si ya conozco el rank de prefijos de longitud len , el prefijo de longitud $2*len$ de sufijo i queda representado por el par $(rank[i], rank[i+len])$. Ordenar esos pares da el nuevo orden. Pares iguales reciben mismo rank. Esto funciona por inducción porque comparar dos mitades ordenadas equivale a comparar el bloque completo.

Kasai: construye LCP en $O(n)$ después de SA. Para cada posición i en el texto, miro el sufijo vecino anterior $j = SA[rank[i]-1]$. Si el LCP anterior era h , al pasar a $i+1$ el LCP baja a lo mucho en 1, así que empiezo en $\max(h-1, 0)$ y avanzo mientras coincidan caracteres. Esto evita recomparar todo.

Formula clave: si $rank[i] = a < b = rank[j]$, entonces $lcp(i, j) = \min(LCP[a+1..b])$. Por eso se hace RMQ sobre LCP. Esta formula aparece en comparar subcadenas, longest common substring, ordenar substrings y periodicidad.

3. Búsqueda y conteo de subcadenas

Problema tipo: texto fijo T y muchos patrones P. Todos los sufijos que empiezan con P aparecen consecutivos en SA. Se hacen dos binary searches: L = primer sufijo $\geq P$; R = primer sufijo que ya no tiene prefijo P. Si $L=R$ no aparece. Si no, ocurrencias = $R-L$ y posiciones = $SA[L..R-1]$.

Para conteo solo respondes $R-L$. Para búsqueda Yes/No basta revisar si L existe y $has_prefix(T[SA[L]..], P)$. Complejidad simple: construir SA $O(n \log n)$, cada patron $O(|P| \log n)$. Si el examen pide optimizar, se menciona LCP/RMQ para acelerar comparaciones, pero la idea teórica central es el intervalo contiguo.

Correctitud: orden lexicográfico agrupa strings con mismo prefijo porque cualquier sufijo con prefijo P queda entre P y el primer string mayor que todos los que empiezan con P. No puede haber un sufijo sin prefijo P dentro del bloque, porque rompería la propiedad de orden.

4. Subcadenas distintas, LCS y ordenar subcadenas

Subcadenas distintas: cada subcadena es prefijo de algun sufijo. El sufijo $SA[i]$ aporta $n - SA[i]$ prefijos, pero los primeros $LCP[i]$ ya aparecieron con el sufijo anterior. Respuesta: $n(n+1)/2 - \sum LCP[i]$.

Longest common substring de A y B: construye $S = A\#B\$$ con separadores que no aparecen. En SA, mira pares vecinos de origen distinto; el mayor LCP entre ellos es una subcadena común máxima. Si piden lexicográficamente menor en empate, entre candidatas de misma longitud compara por $rank/posición$ lexicográfica.

Ordenar subcadenas $(l1, r1)$ vs $(l2, r2)$: $len1, len2$. $x = \min(lcp(l1, l2), len1, len2)$ usando RMQ sobre LCP. Si $x < \min(len1, len2)$, compara $S[l1+x]$ y $S[l2+x]$. Si una se acaba, la más corta es menor. Si son iguales como palabra, desempata por el par original (l, r) si el statement lo pide. Complejidad de comparación $O(1)$ después de SA+LCP+RMQ; ordenar q substrings $O(q \log q)$.

5. Imperial Roads - MST forzando una arista

Problema: grafo conectado ponderado. Cada query exige incluir una arista $e=(u,v,w)$. Queremos costo minimo de un arbol conexo que incluya e . Solucion: calcula MST T con peso W . Si agregas e a T , aparece un unico ciclo: $e + \text{camino}_T(u,v)$. Para volver a arbol y minimizar costo, quitas la arista mas pesada de ese camino. Respuesta $W + w - \text{maxEdgePath}(u,v)$.

Estructura para maxEdgePath : root al MST y binary lifting guardando $\text{up}[v][j]$ y $\text{mx}[v][j] = \text{maximo peso al subir } 2^j$. Para query, igualas profundidades y subes ambos nodos acumulando maximos hasta LCA.

Correctitud: por propiedad del ciclo del MST, en cualquier ciclo se puede eliminar una arista maxima sin empeorar. Al forzar e , todos los arboles que contienen e se obtienen rompiendo ese ciclo; quitar la maxima minimiza el nuevo peso. Complejidad: Kruskal $O(m \log m)$, preprocess $O(n \log n)$, query $O(\log n)$.

6. Fools and Roads - contar caminos que usan cada arista

Problema: arbol y muchas consultas (u,v) . Para cada arista, contar cuantas rutas $u-v$ pasan por ella. No recorras cada camino. Usa diferencia en arbol: $\text{cnt}[u]++$, $\text{cnt}[v]++$, $\text{cnt}[\text{lca}(u,v)]-=2$. Luego haces DFS postorder; al acumular $\text{sumSub}[x]$, ese valor es cuantas rutas cruzan la arista $\text{parent}[x]-x$.

Por que funciona: una ruta $u-v$ sube desde u al lca y baja a v . Al poner $+1$ en extremos y -2 en el lca, el flujo acumulado sube exactamente por las aristas del camino y se cancela fuera del camino. Complejidad: preprocess LCA $O(n \log n)$, cada query $O(\log n)$, DFS $O(n)$.

7. Practica profesor - subsecuencia en camino de arbol

Enunciado resumido: cada vertice tiene una letra. Query U,V,S . Se forma T con las letras del camino simple $U \rightarrow V$ en orden. Decidir si S es subsecuencia de T .

Solucion de examen: HLD descompone el camino $U \rightarrow V$ en $O(\log n)$ segmentos de cadenas pesadas. En el arreglo base de HLD, para cada caracter c guardo la lista ordenada de posiciones donde aparece c . Para consumir S , recorro los segmentos del camino en el orden real: primero $U \rightarrow \text{LCA}$ (direccion hacia arriba, o sea posiciones decrecientes dentro de cada chain), luego $\text{LCA} \rightarrow V$ (direccion hacia abajo). En cada segmento intento ubicar el siguiente caracter pendiente con binary search en la lista de ese caracter, respetando limites y direccion. Si aparece, avanzo el puntero en el segmento y consumo el caracter; si no aparece, paso al siguiente segmento. Si consumo todo S , responde YES.

Correctitud: HLD no cambia el orden del camino; solo lo parte en segmentos contiguos. Las listas por caracter encuentran la primera aparicion valida dentro del segmento. Consumir siempre la aparicion mas temprana es optimo para subsecuencia: deja el mayor sufijo posible del camino para los caracteres restantes. Complejidad: $O(\log n)$ segmentos y cada caracter consumido una vez con busqueda $O(\log n)$: $O((|S| + \log n) \log n)$ por query, espacio $O(n + 26n \text{ conceptual})$.

8. Practica profesor - ordenar nodos de trie

Cada nodo representa la cadena formada desde la raiz hasta ese nodo. Para ordenar lexicograficamente todos los nodos, haz DFS desde la raiz visitando primero el nodo actual y luego sus hijos en orden de caracter. Si root representa cadena vacia, va primero.

Por que funciona: en orden lexicografico, una palabra aparece antes que cualquier extension propia; por eso visitamos el nodo antes que sus hijos. Entre ramas con primer caracter distinto, el caracter menor debe aparecer antes; por eso hijos ordenados. Si hay alfabeto lowercase, se puede iterar 'a'..'z' y queda $O(n \sigma)$ o $O(n)$ si $\sigma=26$ constante. Si hijos vienen desordenados, ordenar listas cuesta $O(n \log \sigma)$ o $O(n \log n)$ peor, aun menor que $O(n^2)$.

9. Practica profesor - caminos con peso $\leq C$ en arbol

Interpretacion natural: contar pares de vertices (u,v) cuyo camino tiene peso total $\leq C$. En un arbol hay unico camino. La solucion subcuadratica clasica es centroid decomposition.

En cada centroid c , recolecta distancias desde c hacia los nodos de cada subarbol. Con una lista global D de distancias $\leq C$, ordena D y cuenta pares con suma $\leq C$ usando two pointers. Eso cuenta caminos que pasan por c . Pero pares dentro del mismo subarbol no deberian contarse en este nivel, asi que para cada subarbol se resta el conteo interno de sus distancias. Luego recursas en cada componente al quitar c .

Correctitud: todo camino tiene un primer centroid de la recursion que separa sus endpoints o es uno de ellos; se cuenta exactamente en ese nivel y no en otro. Complejidad: cada nivel procesa listas con sort/two pointers; $O(n \log^2 n)$ simple, $O(n \log n)$ si se optimiza merge/sort. En papel $O(n \log^2 n)$ ya cumple menor que $O(n^2)$.

```
solve(component):
    c = centroid(component)
    ans += count_pairs(all distances from c <= C)
    for each child part p:
        ans -= count_pairs(distances from c inside p)
    recurse on each part
```

10. Dynamic connectivity offline - segment tree sobre tiempo + rollback DSU

Problema tipo: operaciones add edge, remove edge, query connected/component sum. Si puedes leer todas las operaciones antes de responder, es offline. Convierte cada arista en intervalos de vida $[l, r]$ donde esta activa. Inserta cada intervalo en un segment tree de tiempo.

DFS sobre el segment tree: al entrar a un nodo, aplicas union de todas las aristas asignadas a ese intervalo. En hoja t , el DSU contiene exactamente las aristas activas en t , respondes queries. Al salir, haces rollback al snapshot previo. DSU rollback guarda en stack los cambios de parent/size/sum. No uses path compression porque es difícil deshacer; union by size basta.

Correctitud: un intervalo de vida se coloca en nodos que cubren exactamente sus tiempos activos. El camino raíz-hoja t contiene exactamente los intervalos que cubren t . Rollback restaura el estado para no contaminar otra rama. Complejidad: cada intervalo entra en $O(\log q)$ nodos; $O((m \log q) \alpha / \log n)$ según DSU sin compression) y memoria $O(m \log q)$.

```
intervals = map edge -> [start, end]
for each interval [l, r] of edge e: add e to segtree_time nodes
dfs(node):
    snap = dsu.snapshot()
    for e in node.edges: dsu.union(e.u, e.v)
    if leaf: answer queries at time leaf
    else: dfs(left); dfs(right)
    dsu.rollback(snap)
```

11. Dynamic forest conceptual - Euler-tour tree vs Link-cut tree

Si el grafo dinámico siempre es bosque y piden link/cut/connected: Euler-tour tree representa cada árbol por su tour Euler dentro de un BST balanceado. Link y cut se vuelven split/merge de secuencias. Es natural para conectividad, size y agregados de componente/subárbol.

Si piden agregados en camino de un árbol dinámico: Link-cut tree. Idea: partir el árbol en preferred paths, cada path se guarda en un splay auxiliar. $\text{access}(v)$ convierte el camino raíz- v en una preferred path; con makerooot /access puedes exponer $u-v$ y leer el agregado en la raíz del splay. Operaciones link/cut/path query $O(\log n)$ amortizado.

Regla rápida: árbol fijo + path query $\rightarrow$ HLD. Árbol cambia + path query $\rightarrow$ Link-cut. Árbol cambia + componente/subtree $\rightarrow$ Euler-tour tree. Grafo general offline $\rightarrow$ rollback DSU. Solo inserciones $\rightarrow$ DSU.

12. Integer - predecessor y modelos

Problema predecessor: mantener conjunto S de enteros de w bits; insert/delete/member/predecessor/successor. Variables: n número de llaves, $u=2^w$ universo, w bits por palabra. En modelo comparaciones, BST da $O(\log n)$. En word RAM puedes usar bits.

vEB: divide universo en high/low de tamaño $\sqrt{u}$. Guarda min, max, summary de clusters no vacíos y clusters. Recurrencia $T(u)=T(\sqrt{u})+O(1)=O(\log \log u)=O(\log w)$. Problema: espacio $\Theta(u)$ si no se comprime.

Y-fast: combina hashing de prefijos (estilo X-fast) con buckets balanceados de tamaño $O(w)$. Tiene espacio $O(n)$ y operaciones esperadas/amortizadas $O(\log w)=O(\log \log u)$. Respuesta teórica segura cuando el universo es grande y no quieres espacio u .

Fusion tree: en word RAM fuerte, usa branching alto $B=w^{1/5}$; dentro de un nodo compara varias llaves en paralelo con sketches de bits críticos. Altura $O(\log_w n)$, predecessor teórico rápido. Menciona condición de modelo: algunas versiones usan multiplicación; en AC0 RAM hay cuidado.

13. Integer sorting - counting/radix/signature

La cota $\Omega(n \log n)$ es solo para comparaciones. Si las llaves son enteros puedes mirar dígitos. Counting sort cuesta $O(n+U)$ para universo U del dígito. Radix sort parte cada llave en dígitos de b bits y hace counting estable por cada dígito. Si $b=\Theta(\log n)$, cada pasada cuesta $O(n)$ porque hay $n^{\Theta(1)}$ valores; número de pasadas = $\text{ceil}(w/b)$.

Si $w=O(\log n)$, radix/counting da $O(n)$. Si w es mayor, signature sort reduce llaves a firmas/hasheos de $O(\log n)$ bits por niveles y luego usa packed/radix. Para examen suele bastar explicar: romper comparaciones usando representación binaria, elegir dígitos de $\log n$ bits, estabilidad y complejidad $O(n \cdot w / \log n)$.

14. Mini bloque de metricas por si aparece

Una métrica d cumple: no negatividad $d(x,y) \geq 0$, identidad $d(x,y)=0$ iff $x=y$, simetría $d(x,y)=d(y,x)$, desigualdad triangular $d(x,z) \leq d(x,y)+d(y,z)$. Suma de métricas: si d_1 y d_2 son métricas, $d_3=d_1+d_2$ es métrica porque cada propiedad se preserva sumando; triangular sale de $d_1(x,z)+d_2(x,z) \leq d_1(x,y)+d_1(y,z)+d_2(x,y)+d_2(y,z)$.

Para verificar funciones: cuadrados suelen fallar triangular, por ejemplo $d(0,2)=4$ pero $d(0,1)+d(1,2)=2$. $|x-y|$, L_1 , L_2 , L_∞ y pesos positivos de L_1 son métricas. $\sqrt{|x-y|}$ es métrica porque $\sqrt{a+b} \leq \sqrt{a}+\sqrt{b}$. $\min(1, |x-y|)$ también es métrica por truncamiento subaditivo.

Funciones con mezcla $|x-y|+|f(x)-f(y)|$ son metricas si f no destruye identidad; como incluye $|x-y|$, identidad queda.